

Due:03/03/2026

You should submit a typeset or *neatly* written pdf on Canvas. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments.

1. **Short answer** (4 points)

- Consider the CFG:

$$A \rightarrow SA|b$$

- $$S \rightarrow AS \rightarrow SAS \rightarrow SAAS \rightarrow aAAS \rightarrow abAS \rightarrow abbS \rightarrow abba$$

```

graph TD
    S1((S)) --> A1((A))
    S1 --> S2((S))
    A1 --> S3((S))
    A1 --> A2((A))
    S2 --> A3((A))
    S2 --> S4((S))
    S3 --> A4((A))
    S3 --> S5((S))
    A2 --> b1((b))
    A3 --> b2((b))
    A4 --> b3((b))
    A5((A)) --> a1((a))
    S4 --> b4((b))
    S5 --> b5((b))
    S6((S)) --> a2((a))
  
```

- 3: Context-Free languages-1

Answer:In Chomsky normal form each production can either point to a pair of non-terminals or a single terminal. Now given a string of length n say with some arbitrary number of steps / productions, to get a string with length $n + 1$, one of the terminals must be a non-terminal that branches out to generate 2 terminals. Note that we're removing 1 step (step to terminal) adding a step (step to 2 non-terminals) and adding 2 steps (a step each to a terminal for each of those 2 non-terminals). Mathematically this can be seen as,

$$P(n) = P(n - 1) + 2$$

Now for a string of length 1, clearly we just have one step i.e one production. So we have, $P(1) = 1, P(2) = P(1) + 2 = 3, \dots$. So for an arbitrary n we can solve this recursion by expanding it as follows,

$$\begin{aligned} P(n) &= P(n - 1) + 2 \\ &= P(n - 2) + 2 + 2 \\ &= \dots \\ &= P(1) + (n - 1) \cdot 2 \\ &= 1 + 2n - 2 = 2n - 1 \end{aligned}$$

So for a given string of length n the total steps are $2n - 1$

- (b) (1 point - Both) Let us define a context-free grammar G to be in “2-Form” if, for every nonterminal A , there are at most two options for replacing A in a derivation. That is, there are at most two rules in G that have A on the left hand side, where the “rule” $A \rightarrow B|C|D$ is considered to be three rules. Show that, for any context-free grammar, there is a 2-Form context-free grammar that recognizes the same language.

Answer:First consider our grammar G . Now let A be an arbitrary non-terminal in this grammar that have more than two rules in G . So say we have $A \rightarrow X_1 | X_2 | \dots | X_n$, note here A is an arbitrary non-terminal that has n options. Now simply define a new non-terminal $A_2 \rightarrow X_2 | \dots | X_n$. So we have, $A \rightarrow X_1 | A_2$ and $A_2 \rightarrow X_2 | \dots | X_n$, now A_2 is a non-terminal with $n - 1$ productions. So essentially we showed that for any arbitrary non-terminal A with n options we can write it as a pair of productions such that the maximum options are $n - 1$. Inductively we can continue doing this $n - 2$ times so that we end up with $n - (n - 2) = 2$ maximum productions. Hence we have a set of productions for A such that the maximum options for each production is at most 2. We can do this reduction for any arbitrary A and n in our grammar and hence we get a new grammar that accepts the same language but is 2-Form.

- (c) (2 points - Section B) Let G be a context-free grammar for a language L , and suppose L contains only strings of length 2 or greater. Prove that there is a context-free grammar G' which generates L such that every rule of G' has the form $A \rightarrow x_1 x_2$, where A is a nonterminal, and each x_i is a terminal or nonterminal.

Answer:

- (d) (2 points - Section X) Consider a context-free grammar in Chomsky normal form which has n variables. Prove that if there is some string $s \in L(G)$ such that the derivation of s has at least 2^n steps, then $L(G)$ is infinite. (Hint: Think about the parse tree for s)

Answer:In Chomsky normal form each variable / non-terminal goes to either two non-terminals or a terminal. Now consider the binary parse tree. Now in the parse tree at each level the number of leaves is at most 2^h (and 2^h for a perfect tree) where h is the level / height. Now we know from (a) that if we have a length k string then we must have $2k - 1$ steps. So if our string s has at least 2^n steps, this means we have $2k - 1 \geq 2^n$ which gives us $k \geq 2^{n-1} + 1/2$ which means $k \geq 2^{n-1} + 1$ as k must be an integer. This means the length of our string is greater than equal to $2^{n-1} + 1$. Now note that the smallest height tree we can have given a fixed number of leaves is a complete tree. Now in a perfect tree of height h we have 2^h now the minimum height we can have is h so that $2^h \geq 2^{n-1} + 1$. Now note that the power of 2 that is greater than 2^{n-1} is only 2^n . So we have $h \geq n$. Where n is the number of variables. Note that h starts from 0, so at a height of n there exists $n + 1$ nodes, but now note that if we have $n + 1$ nodes for at least one path in our tree but only n variables, this means that there is a repetition. In other words there is some X such that $X \rightarrow^+ X$ i.e some path to get from X to itself. But if there is such a path then this means that we can go down this path an infinite number of times as we can repeat X to itself infinite number of times, each time we repeat it corresponds to a new string in our language. This means our language is infinite.

2. Context Free Grammars (4 points)

Give context free grammars for the following languages:

- (Section B) $L_1 = \{a^i b^j c^k d^l \mid i = j \text{ or } k = l \text{ but not both, } i, j, k, l \geq 0\}$

Answer:

- (Both) $L_2 = \{1^i 0^j \mid \text{for some nonnegative } x, y \in \mathbb{Z}, x + y = i \text{ and } 5x + 2y = j\}$

Answer:To solve this we can note two things. Every time i is incremented by 1, either x is incremented by 1 or y is. But if x is incremented by 1 then j will be 5 as $y = 5x + 2y$ and similarly if y is incremented by 1 then j will be incremented by 2. So all in all we have two choices to take. The grammar would look like,

$$S \rightarrow AB$$

$$A \rightarrow 1^1 A 0^5 \mid \varepsilon$$

$$B \rightarrow 1^1 B 0^2 \mid \varepsilon$$

- (Section X) $L_3 = \text{the complement of } \{ww \mid w \in \Sigma^*\}$.

Answer:Consider for simplicity we have $\Sigma = \{a, b\}$, now we will generate a CFC for L_3 . First we generate all odd length strings centered on a and b . Trivially, this is in L_3 because we can't write it as ww where $w \in \Sigma^*$. So consider the following CFG,

$$\begin{aligned}
S &\rightarrow A \mid B \\
A &\rightarrow aAa \mid aAb \mid bAb \mid bAa \mid a \\
B &\rightarrow aBa \mid aBb \mid bBb \mid bBa \mid b
\end{aligned}$$

Note that this grammar gives us all odd length strings. A allows for any combination of strings on the outside and the only non-terminal is a which will have to end up in the center, similar for B .

Now we have to generate even length strings that are not the concatenation of two equal strings. So consider the strings AB and BA clearly as A and B are odd length their concat are even length. So our new grammar is,

$$\begin{aligned}
S &\rightarrow A \mid B \mid AB \mid BA \\
A &\rightarrow aAa \mid aAb \mid bAb \mid bAa \mid a \\
B &\rightarrow aBa \mid aBb \mid bBb \mid bBa \mid b
\end{aligned}$$

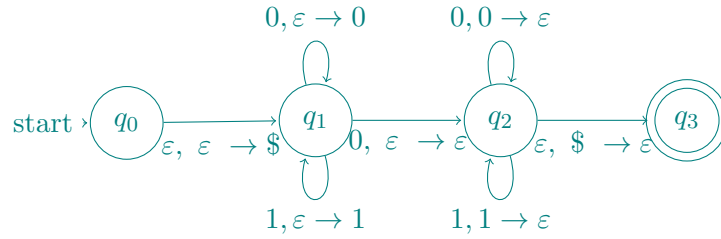
But we still need to show that their concatenation could never be split into two strings that are equal. So WLOG consider AB where $|A| = 2m + 1$ and $|B| = 2n + 1$ and $m < n$. Now what we need to ensure is that the centers of A and centers of B are NON-equal as we know A and B are centered by a and b respectively. So in AB the center of A i.e a is at position $m + 1$ and for B is $n + 1$. Now the absolute center of the string AB is at $(2m + 1 + 2n + 1)/2 = m + n + 1$. Now consider our string is wv where $|w| = |v|$ then v starts at position $m + n + 2$. Now relative to w the center of A is at position $m + 1$. And relative to v the center of B would be at the absolute position of the center - middle of the string. The absolute position of the center of B is at $2m + 1 + (n + 1)$ and the middle as we know is $m + n + 1$ so our relative position is $2m + 1 + n + 1 - m - n - 1 = m + 1$. Note now that at position $m + 1$ for string w we have the center of A which is a and at position $m + 1$ of string v we have the center of B which is b . But clearly as $|w| = |v|$ this means that the string w can never be equal to v as we just showed that the position $m + 1$ will always be unequal. This means we can never have ww as a part of our language.

3. Pushdown Automata ***Both Sections***(4 points)

Give pushdown automata for the following languages. You may assume $\Sigma = \{0, 1\}$.

- $L_4 = \{w0w^R \mid w \in \Sigma^*\}$, the set of palindromes with a 0 as the middle character

Answer:Our PDA is as follows,

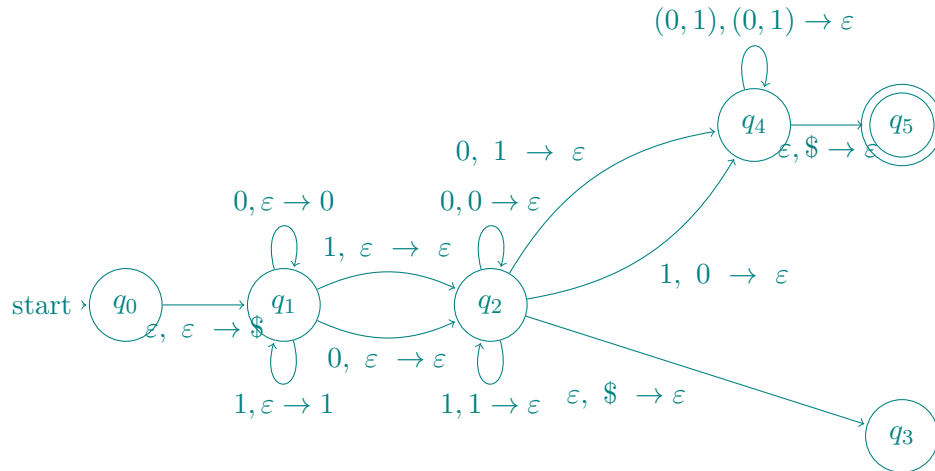


Note that first from $q_0 \rightarrow q_1$ we have an ε transition where we append \$ to the stack that will represent the bottom of the stack. Now we will start consuming w . Note that at q_1 we have two high level choices, one is to consume w and one is to guess the middle 0 and leave. So for consuming w for any alphabet $a \in \{0, 1\}$ we push a onto the stack when we encounter it. Now when we encounter the middle 0 we don't push or pop the stack and just move to the state q_2 which represents the reverse string. At q_2 now note that both our non-epsilon transitions are dependent on the stack top. So if we get a 1 and a 1 is at the top then we pop and we do the same for for a 0. This ensures that the sequence of letters we get are the reverse of what was pushed to the stack. Clearly if we have either 1, 0 or 0, 1 we have no where to go and naturally we don't accept the string. Now if we ever pop every alphabet from Σ off the stack we will be left with \$ for which we have an ε transition that will go to q_3 the accepting state.

- $L_5 = \{w \mid w \text{ is odd length but is not a palindrome}\}$

Answer:

We have two conditions, first w is odd length and second w is not a palindrome. Now if w is odd length then it means we can split w as axb where $|a| = |b|$ and $x \in \Sigma$. So we can ensure it's odd length by guessing x in our PDA and moving to another state where we implicitly count b . Secondly, to ensure it's not a palindrome we a to our stack guess x and start popping b from the stack, if it ever doesn't match then we move to a state where we just count the rest of the letters and if it's equal in length we just accept, if it matches all the way to the bottom then we know that it's a palindrome and hence we reject. The PDA looks like,



So first we add a \$ to the stack to represent the bottom. Now for each string with 0 and 1 the state q_1 will add the symbol to the stack, then we will guess the center symbol, be it 0 or 1 without changing the stack and move to state q_2 which represents the second half after the middle. At q_2 we have three high level cases, first we match the alphabet in the top of the stack, in this case we just pop the stack and continue to stay in q_2 , second if we don't match the top of the stack we move to state q_4 and third we reached the end of the stack. Now note if we reach the end of the stack the second half matched perfectly with the stack's pops which means the string is a palindrome and as q_3 is a sink state we reject automatically. Now, going back to the second case if we don't match then we know we can never have a palindrome but we still need to count whether we have the same number of symbols as in the stack (if we have the same it implies we guessed the middle right and that we have an odd length string), so at q_4 for any symbol $a \in (0, 1)$ for any top of the stack (if it's 0 or 1), we pop it (note that if we guess the middle wrong early, then we end up popping everything in the stack until the \$ at which point we don't have any transitions and hence reject). The point is we just want to count the symbols so it's enough to just pop the stack until we reach the bottom.

4. Pumping Lemma ***Both Sections*** (4 points)

Show that $\{x_1 \star x_2 \star \dots \star x_t \mid t > 1, x_i \in \{a, b\}^*, \text{ and } \exists x_i, x_j \text{ with } i \neq j \text{ but } x_i = x_j\}$ is not context-free using the pumping lemma for context free languages.

Answer: First we assume that it is context free and take the pumping length to be p , so we know that for any string $s \in L$ such that $|s| > p$ we have $s = uvxyz$ such that all $uv^i xy^i z \in L$ while $|vy| > 0$ and $|vxy| \leq p$.

Now choose our string $s = a^p b^p \star a^p b^p$. Now we need to consider all possible $|vxy|$ such that the length is smaller than p . So note for all substrings of length smaller than p we have three choices (note these choices are exhaustive, i.e it covers all cases),

- (a) vxy is a substring of x_1 , so this means that it's a part of the first string.

Now in this case if $vxy \subset x_1$, then note that for any choice of vy such that $|vy| > 0$, if we pump v and y then we trivially increase (or decrease if $i = 0$) the size of the string x_1 . However this means that we can never have $x_1 = x_2$ because $|x_1| \neq |x_2|$.

- (b) vxy exists in the second half i.e a substring of x_2 .

Similar to case (a) we have size of x_2 can be pumped to be greater than size of x_1 which will trivially make it non-equal and out of our language.

- (c) vxy exists in the middle, so this means we have $\star$ a substring of vxy . Now

In this case note that our string is $a^p b^p \star ba \star a^p b^p$ and we know our string is length smaller than equal to p and contains $\star$. So the only possible way is if we have $vxy = b^i \star a^j$ where $i + j < p$. Now note we have different possibilities for v, x, y for this. Consider two cases that are exhaustive, (1) v is entirely a substring of b^i and let xy be anything. Here if we pump v note that we change the total number of b in the string x_i but xv does NOT contain any b so we can never change the number of b 's in string x_2 no matter our choice of y . So this necessarily means $x_1 \neq x_2$ for all choices of x and y . (2) We have v is entirely in the second half i.e a^j , we have have a similar problem as (1) as we can increase the number of a' in x_2 while not being able to change anything of x_1 hence $x_1 \neq x_2$. (3) Last case is when v contains the $\star$, now in this case note that no matter the configuration of v, x or y if we choose $i = 0$ we necessarily have the $\star$ disappear as we have v^0 and v contains the only $\star$. But note that our language necessarily has $t > 1$ which means that a string $x_i \in \{a, b\}^*$ is not accepted by the language.

Hence, we just showed for any valid decomposition we have a contradiction.